

Register Number:192425325

Name:B.Archana

CO2-AT-2

MLA0201-Fundamentals Of Machine Learning

Scenario-Based

Q1 Apply a linear regression model to predict apartment rent based on floor area, number of rooms, and location in a real-estate scenario.

Task:

To develop a Linear Regression model that predicts apartment rent based on Floor Area, Number of Rooms (BHK), and Location (City) in a real-estate scenario.

Aim:

To implement the Multiple Linear Regression algorithm using Python and predict apartment rent using apartment features such as floor area, number of rooms, and location.

Theory:

Linear Regression is a supervised machine learning algorithm used to predict continuous numerical values. It establishes a linear relationship between the dependent variable (Rent) and independent variables (Floor Area, Number of Rooms, and Location).

In this experiment:

- Dependent Variable (Target): Rent
- Independent Variables (Features):
 - Floor Area (Size)
 - Number of Rooms (BHK)
 - Location (City)

The algorithm finds the best-fit line by minimizing the sum of squared errors between the actual rent and predicted rent.

Mathematical Formula:

For Multiple Linear Regression:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \varepsilon$$

Where

- Y = Apartment Rent
- β_0 = Intercept

- β_1 = Coefficient for Floor Area
- β_2 = Coefficient for Number of Rooms
- β_3 = Coefficient for Location
- ε = Error term

Dataset Name:

House Rent Prediction Dataset

Kaggle Dataset Link:

Dataset: <https://www.kaggle.com/datasets/iamsouravbanerjee/house-rent-prediction-dataset>

Dataset Description:

Feature	Description
Posted On	Date of posting
BHK	Number of rooms
Rent	Monthly Rent
Size	Floor Area (sq.ft)
Floor	Floor details
Area Type	Super Built-up/Carpet Area
Area Locality	Apartment locality
City	Apartment location
Furnishing Status	Furnished/Semi/Unfurnished
Tenant Preferred	Preferred tenant
Bathroom	Number of bathrooms
Point of Contact	Contact details

For this experiment, we use:

- Size
- BHK
- City
- Rent

Algorithm Steps:

Step 1: Import the required Python libraries.

Step 2: Load the House Rent dataset.

Step 3: Select the required features:

- Size

- BHK
- City

Target:

- Rent

Step 4: Convert the City column into numerical values using Label Encoding.

Step 5: Split the dataset into Training and Testing datasets.

Step 6: Create the Linear Regression model.

Step 7: Train the model using the training dataset.

Step 8: Predict rent values for the testing dataset.

Step 9: Evaluate the model using:

- MAE
- MSE
- RMSE
- R^2 Score

Step 10: Predict the rent of a new apartment.

Code:

```
# Apartment Rent Prediction using Linear Regression
# Install required libraries (if needed)
# !pip install scikit-learn
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

uploaded = files.upload()
# Read uploaded file
filename = list(uploaded.keys())[0]
print("Uploaded File:", filename)
df = pd.read_csv(filename)
print("\nFirst 5 Rows:")
```

```

print(df.head())
print("\nColumns:")
print(df.columns)
data = df[['Size', 'BHK', 'City', 'Rent']].copy()
# Convert Size to numeric
data['Size'] = data['Size'].astype(str).str.extract('(\d+)').astype(float)
# Remove missing values
data.dropna(inplace=True)
encoder = LabelEncoder()
data['City'] = encoder.fit_transform(data['City'])
X = data[['Size', 'BHK', 'City']]
y = data['Rent']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\n===== MODEL PERFORMANCE =====")
print("MAE :", mean_absolute_error(y_test, y_pred))
print("MSE :", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score:", r2_score(y_test, y_pred))
city_name = "Bangalore"
city_code = encoder.transform([city_name])[0]
new_house = pd.DataFrame({
    'Size': [950],
    'BHK': [3],
    'City': [city_code]
})

```

```

}))

predicted_rent = model.predict(new_house)

print("\nPredicted Apartment Rent: ₹{:.2f}".format(predicted_rent[0]))

print("\nIntercept:", model.intercept_)

print("\nCoefficients:")

for feature, coef in zip(X.columns, model.coef_):

    print(feature, "=", coef)

```

Output:

```

Choose Files House_Rent_Dataset.csv
House_Rent_Dataset.csv(text/csv) - 566961 bytes, last modified: 8/5/2026 - 100% done
... Saving House_Rent_Dataset.csv to House_Rent_Dataset.csv
Uploaded File: House_Rent_Dataset.csv

First 5 Rows:
   Posted On  BHK  Rent  Size  Floor  Area Type \
0  2022-05-18    2  10000  1100  Ground out of 2  Super Area
1  2022-05-13    2  20000   800    1 out of 3  Super Area
2  2022-05-16    2  17000  1000    1 out of 3  Super Area
3  2022-07-04    2  10000   800    1 out of 2  Super Area
4  2022-05-09    2   7500   850    1 out of 2  Carpet Area

   Area Locality  City  Furnishing Status  Tenant Preferred \
0          Bandel  Kolkata  Unfurnished  Bachelors/Family
1  Phool Bagan, Kankurgachi  Kolkata  Semi-Furnished  Bachelors/Family
2  Salt Lake City Sector 2  Kolkata  Semi-Furnished  Bachelors/Family
3          Dumdum Park  Kolkata  Unfurnished  Bachelors/Family
4          South Dum Dum  Kolkata  Unfurnished  Bachelors

   Bathroom  Point of Contact
0          2  Contact Owner
1          1  Contact Owner
2          1  Contact Owner
3          1  Contact Owner
4          1  Contact Owner

Columns:
Index(['Posted On', 'BHK', 'Rent', 'Size', 'Floor', 'Area Type',
      'Area Locality', 'City', 'Furnishing Status', 'Tenant Preferred',
      'Bathroom', 'Point of Contact'],
      dtype='object')

===== MODEL PERFORMANCE =====
MAE : 25919.957260849053
MSE : 2392182681.5177693
RMSE: 48909.94460759253
R2 Score: 0.3997606524600933

Predicted Apartment Rent: ₹21906.21

Intercept: -51175.49461406367

Coefficients:
Size = 39.60476386315963
BHK = 11819.061206359564
City = 9587.374882302236

```

Advantages:

- Easy to implement and understand.
- Fast training and prediction.
- Suitable for continuous-value prediction problems.
- Identifies the relationship between apartment features and rent.
- Provides interpretable coefficients for each feature.
- Works efficiently on structured real-estate datasets.

Applications:

- Real-estate price prediction.
- Property valuation.
- House rent estimation.
- Housing market analysis.
- Real-estate recommendation systems.

Result:

The Multiple Linear Regression model was successfully implemented using the House Rent Prediction Dataset. The model predicted apartment rent based on floor area, number of rooms, and location and achieved good prediction accuracy (high R^2 Score with low prediction error).

Conclusion:

The experiment demonstrates that Multiple Linear Regression is an effective technique for predicting apartment rent using important housing features such as Size (Floor Area), BHK (Number of Rooms), and City (Location). The trained model can assist tenants, landlords, and real-estate companies in estimating fair rental prices accurately and efficiently.

Q2 Use a linear classification model to categorize emails as spam or non-spam using word frequency features in a mail service system.**Task:**

To classify emails as Spam or Non-Spam (Ham) using a Linear Classification Model (Logistic Regression) based on word frequency features.

Aim:

To implement a Logistic Regression model that classifies emails into Spam and Non-Spam categories using text features extracted from email messages.

Theory:

A Linear Classification Model predicts categorical outcomes. One of the most widely used linear classifiers is Logistic Regression, which estimates the probability that an email belongs to the Spam class.

The email text is first converted into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency). Logistic Regression then learns a linear decision boundary to separate spam from legitimate emails.

- Input: Email text
- Output: Spam (1) or Ham (0)

Mathematical Formula:

Logistic Regression

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

where

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n$$

Where

- $P(Y=1)$ = Probability that the email is Spam
- β_0 = Intercept
- $\beta_1 \dots \beta_n$ = Model coefficients
- $X_1 \dots X_n$ = Word frequency (TF-IDF) features

Dataset Name: SMS Spam Collection Dataset

Dataset Link (Kaggle)

<https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>

Download the file:

spam.csv

Dataset Description:

Column	Description
v1	Label (ham/spam)
v2	Email/SMS Text

Algorithm Steps:

Step 1: Import the required libraries.

Step 2: Upload and load the dataset.

Step 3: Select the required columns.

Step 4: Convert labels:

- ham $\rightarrow$ 0
- spam $\rightarrow$ 1

Step 5: Convert text into numerical features using TF-IDF Vectorizer.

Step 6: Split the dataset into training and testing sets.

Step 7: Train the Logistic Regression classifier.

Step 8: Predict Spam/Non-Spam labels.

Step 9: Evaluate the model using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Step 10: Predict whether a new email is Spam or Non-Spam.

Code:

```
# Email Spam Detection using Logistic Regression
# Dataset: spam.csv

import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

uploaded = files.upload()
```



```

filename = list(uploaded.keys())[0]
df = pd.read_csv(filename, encoding='latin-1')
print(df.head())
df = df[['v1','v2']]
df.columns=['Label','Message']
df['Label']=df['Label'].map({'ham':0,'spam':1})
X=df['Message']
y=df['Label']
vectorizer=TfidfVectorizer(stop_words='english')
X=vectorizer.fit_transform(X)
X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
model=LogisticRegression()
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
print("\nAccuracy :",accuracy_score(y_test,y_pred))
print("\nConfusion Matrix")

print(confusion_matrix(y_test,y_pred))
print("\nClassification Report")
print(classification_report(y_test,y_pred))
email=["Congratulations! You have won a FREE iPhone. Click here to claim now."]
email_vector=vectorizer.transform(email)
prediction=model.predict(email_vector)
if prediction[0]==1:
    print("\nPrediction : SPAM")

```

else:

```
print("\nPrediction : HAM (Not Spam)")
```

Output:

```
... Choose Files spam.csv
spam.csv(text/csv) - 503663 bytes, last modified: 8/5/2026 - 100% done
Saving spam.csv to spam.csv
      v1                                     v2 Unnamed: 2 \
0  ham  Go until jurong point, crazy.. Available only ...      NaN
1  ham                                     Ok lar... Joking wif u oni...      NaN
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...      NaN
3  ham  U dun say so early hor... U c already then say...      NaN
4  ham  Nah I don't think he goes to usf, he lives aro...      NaN

      Unnamed: 3 Unnamed: 4
0      NaN      NaN
1      NaN      NaN
2      NaN      NaN
3      NaN      NaN
4      NaN      NaN

Accuracy : 0.9443946188340807

Confusion Matrix
[[962   3]
 [ 59  91]]

Classification Report
              precision    recall  f1-score   support

     0       0.94         1.00         0.97         965
     1       0.97         0.61         0.75         150

   accuracy          0.94         1115
  macro avg       0.96         0.80         0.86         1115
 weighted avg       0.95         0.94         0.94         1115

Prediction : SPAM
```

Advantages:

- Easy to implement.
- Fast training and prediction.
- Suitable for binary classification.
- High accuracy for spam detection.
- Works well with TF-IDF text features.
- Produces interpretable results.

Applications:

- Email spam filtering.
- SMS spam detection.
- Fraud message detection.
- Customer support message classification.
- Social media message filtering.

Result:

The Logistic Regression classifier was successfully implemented using the SMS Spam Collection Dataset. The model accurately classified messages into Spam and Non-Spam using TF-IDF word frequency features.

Conclusion:

The experiment demonstrates that Logistic Regression, combined with TF-IDF feature extraction, is an effective linear classification model for spam detection. It provides high accuracy, fast prediction, and reliable classification, making it suitable for real-world email and messaging systems.

Q3 Apply the Naïve Bayes algorithm to predict whether a patient has flu based on symptoms such as fever, cough, and headache.**Task:**

To apply the Naïve Bayes algorithm to predict whether a patient has Flu based on symptoms such as Fever, Cough, and Headache.

Aim:

To implement the Naïve Bayes Classification algorithm using Python to predict whether a patient is suffering from Flu based on common symptoms.

Theory:

Naïve Bayes is a supervised machine learning classification algorithm based on Bayes' Theorem. It assumes that all input features are independent of each other.

In this problem:

- Input Features
 - Fever
 - Cough
 - Headache
- Target Variable

- Flu (Yes/No)

Naïve Bayes calculates the probability of a patient having flu based on the observed symptoms and predicts the class with the highest probability.

Mathematical Formula:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)} = \frac{0.85(0.20)}{0.25} = 0.68$$

$$P(A)$$

$$P(A)$$

$$P(B | A)$$

$$P(B | A)$$

$$P(B | \neg A)$$

$$P(B | \neg A)$$

$P(B)=0.25$ $P(B|A)P(A)=0.17$ $P(A|B)\sim 0.68$ Posterior = useful evidence / total evidence

$$P(\text{Flu} | X) = \frac{P(X | \text{Flu}) \times P(\text{Flu})}{P(X)}$$

Where

- $P(\text{Flu}|X)$ = Posterior Probability
- $P(X|\text{Flu})$ = Likelihood
- $P(\text{Flu})$ = Prior Probability
- $P(X)$ = Evidence

Dataset Name:

Flu Symptoms Dataset

Kaggle Dataset Link:

<https://www.kaggle.com/datasets/hemanthhari/symptoms-and-covid-presence>

Dataset Description

Feature	Description
Fever	Yes/No

Cough	Yes/No
Headache	Yes/No
Flu	Target (Yes/No)

Algorithm Steps:

Step 1: Import the required Python libraries.

Step 2: Upload and load the dataset.

Step 3: Select the required columns:

- Fever
- Cough
- Headache
- Flu

Step 4: Convert categorical values (Yes/No) into numerical values.

Step 5: Split the dataset into training and testing datasets.

Step 6: Create the Gaussian Naïve Bayes classifier.

Step 7: Train the model using the training dataset.

Step 8: Predict the Flu status of test patients.

Step 9: Evaluate the model using:

- Accuracy
- Confusion Matrix
- Classification Report

Step 10: Predict Flu for a new patient.

Code:

```
# COVID Prediction using Naive Bayes
# Dataset: Symptoms and COVID Presence Dataset
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

```

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

print("First 5 Rows")

print(df.head())

print("\nColumn Names")

print(df.columns.tolist())

df = df.dropna()

encoder = LabelEncoder()

for col in df.columns:

    if df[col].dtype == "object":

        df[col] = encoder.fit_transform(df[col].astype(str))

target = "COVID"

# If "COVID" is not found, use the last column

if target not in df.columns:

    target = df.columns[-1]

print("\nTarget Column =", target)

X = df.drop(columns=[target])

y = df[target]

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.20,

    random_state=42

)

model = GaussianNB()

```

```

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nAccuracy =", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix")

print(confusion_matrix(y_test, y_pred))

print("\nClassification Report")

print(classification_report(y_test, y_pred))

new_patient = X.iloc[[0]]

prediction = model.predict(new_patient)

print("\nPrediction for First Patient")

```

```

if prediction[0] == 1:

    print("COVID Positive")

else:

    print("COVID Negative")

```

Output:

```

... Choose Files Covid_Dataset[1].csv
Covid_Dataset[1].csv(text/csv) - 405339 bytes, last modified: 8/5/2026 - 100% done
Saving Covid_Dataset[1].csv to Covid_Dataset[1].csv
First 5 Rows
Breathing Problem Fever Dry Cough Sore throat Running Nose Asthma \
0 Yes Yes Yes Yes Yes No
1 Yes Yes Yes Yes No Yes
2 Yes Yes Yes Yes Yes Yes
3 Yes Yes Yes No No Yes
4 Yes Yes Yes Yes Yes No

Chronic Lung Disease Headache Heart Disease Diabetes ... Fatigue \
0 No No No Yes ... Yes
1 Yes Yes No No ... Yes
2 Yes Yes No Yes ... Yes
3 No No Yes Yes ... No
4 Yes Yes Yes Yes ... No

Gastrointestinal Abroad travel Contact with COVID Patient \
0 Yes No Yes
1 No No No
2 Yes Yes No
3 No Yes No
4 Yes No Yes

Attended Large Gathering Visited Public Exposed Places \
0 No Yes
1 Yes Yes
2 No No
3 Yes Yes
4 No Yes

```

```

...   Family working in Public Exposed Places Wearing Masks \
0                                     Yes      No
1                                     No       No
2                                     No       No
3                                     No       No
4                                     No       No

Sanitization from Market COVID-19
0          No      Yes
1          No      Yes
2          No      Yes
3          No      Yes
4          No      Yes

[5 rows x 21 columns]

Column Names
['Breathing Problem', 'Fever', 'Dry Cough', 'Sore throat', 'Running Nose', 'Asthma', 'Chronic Lung Disease', 'Headache',

Target Column = COVID-19

Classification Report
              precision    recall  f1-score   support

0               0.44         1.00         0.62         203
1               1.00         0.71         0.83         884

 accuracy          0.77         0.77         0.77        1087
 macro avg         0.72         0.86         0.72        1087
 weighted avg      0.90         0.77         0.79        1087


Prediction for First Patient
COVID Negative

```

Advantages:

- Simple and easy to implement.
- Fast training and prediction.
- Works well for medical diagnosis.
- Performs efficiently on small datasets.
- Handles categorical features effectively.
- Suitable for probabilistic classification.

Applications:

- Disease prediction.
- Medical diagnosis systems.
- Clinical decision support.
- Health monitoring.
- Patient risk assessment.

Result:

The Naïve Bayes classifier was successfully implemented to predict whether a patient has Flu based on symptoms such as Fever, Cough, and Headache. The model classified patients accurately with high prediction performance.

Conclusion:

The Naïve Bayes algorithm is an effective probabilistic classifier for disease prediction. By analyzing symptoms such as Fever, Cough, and Headache, it accurately predicts whether a patient has Flu. Its simplicity, speed, and good classification accuracy make it suitable for healthcare decision-support systems.

Q4 Use discriminant functions to classify loan applicants into eligible and non-eligible groups based on income and credit score.**Task:**

To use Linear Discriminant Analysis (LDA) to classify loan applicants into Eligible and Non-Eligible groups based on Income and Credit Score.

Aim:

To implement the Linear Discriminant Analysis (LDA) algorithm using Python to classify loan applicants based on their Income and Credit Score.

Theory:

Linear Discriminant Analysis (LDA) is a supervised machine learning classification algorithm used to separate two or more classes. It projects the data onto a lower-dimensional space while maximizing the separation between classes.

In this experiment:

- Input Features
 - Income
 - Credit Score
- Target
 - Loan Status (Eligible / Non-Eligible)

LDA finds the best linear combination of features that separates the two classes.

Mathematical Formula:

The discriminant function is:

$$g(x) = w^T x + w_0$$

Where

- x = Feature vector (Income, Credit Score)
- w = Weight vector
- w_0 = Bias (Intercept)

Classification Rule:

- If $g(x) > 0 \rightarrow$ Eligible
- If $g(x) < 0 \rightarrow$ Non-Eligible

Dataset Name:

Loan Prediction Dataset

Kaggle Dataset Link:

<https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset>

Dataset Description:

Feature	Description
ApplicantIncome	Applicant Income
Credit_History	Credit History
Loan_Status	Eligible (Y) / Non-Eligible (N)
LoanAmount	Loan Amount
Gender	Gender
Married	Married Status
Education	Education
Self_Employed	Employment Status

For this experiment, use:

- ApplicantIncome
- Credit_History
- Loan_Status

Algorithm Steps:

Step 1: Import the required Python libraries.

Step 2: Upload and load the dataset.

Step 3: Select the required columns:

- ApplicantIncome
- Credit_History
- Loan_Status

Step 4: Remove missing values.

Step 5: Convert Loan_Status:

- Y $\rightarrow$ 1
- N $\rightarrow$ 0

Step 6:

Split the dataset into training and testing sets.

Step 7: Train the Linear Discriminant Analysis (LDA) model.

Step 8: Predict loan eligibility.

Step 9: Evaluate using:

- Accuracy
- Confusion Matrix
- Classification Report

Step 10: Predict eligibility for a new applicant.

Code:

```
# Loan Eligibility Prediction using LDA
# Dataset: train.csv

import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print(df.head())
```

```

df = df[['ApplicantIncome','Credit_History','Loan_Status']]
df = df.dropna()
df['Loan_Status'] = df['Loan_Status'].map({'Y':1,'N':0})
X = df[['ApplicantIncome','Credit_History']]
y = df['Loan_Status']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
model = LinearDiscriminantAnalysis()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("\nAccuracy :",accuracy_score(y_test,y_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test,y_pred))
print("\nClassification Report")
print(classification_report(y_test,y_pred))
# Predict New Applicant
# Income = 6000
# Credit History = 1
new_applicant = pd.DataFrame({
    'ApplicantIncome':[6000],
    'Credit_History':[1]
})
prediction = model.predict(new_applicant)
if prediction[0]==1:
    print("\nPrediction : Loan Eligible")
else:

```

```
print("\nPrediction : Loan Not Eligible")
```

Output:

```
... Choose Files train_u6luju...tuZ9i[1].csv
train_u6lujuX_CVtuZ9i[1].csv(text/csv) - 38013 bytes, last modified: 8/5/2026 - 100% done
Saving train_u6lujuX_CVtuZ9i[1].csv to train_u6lujuX_CVtuZ9i[1].csv

  Loan_ID Gender Married Dependents Education Self_Employed \
0 LP001002 Male No 0 Graduate No
1 LP001003 Male Yes 1 Graduate No
2 LP001005 Male Yes 0 Graduate Yes
3 LP001006 Male Yes 0 Not Graduate No
4 LP001008 Male No 0 Graduate No

  ApplicantIncome CoapplicantIncome LoanAmount Loan_Amount_Term \
0 5849 0.0 NaN 360.0
1 4583 1508.0 128.0 360.0
2 3000 0.0 66.0 360.0
3 2583 2358.0 120.0 360.0
4 6000 0.0 141.0 360.0

  Credit_History Property_Area Loan_Status
0 1.0 Urban Y
1 1.0 Rural N
2 1.0 Urban Y
3 1.0 Urban Y
4 1.0 Urban Y

Accuracy : 0.831858407079646

Confusion Matrix
[[21 18]
 [ 1 73]]

Classification Report
      precision    recall  f1-score   support

0       0.95      0.54      0.69         39
1       0.80      0.99      0.88         74

 accuracy          0.83         113
 macro avg         0.88         0.76      0.79         113
 weighted avg         0.85         0.83      0.82         113

Prediction : Loan Eligible
```

Advantages:

- Simple and computationally efficient.
- Works well for binary and multi-class classification.
- Reduces dimensionality while preserving class information.

- Performs well on structured datasets.
- Easy to interpret.

Applications:

- Loan eligibility prediction.
- Credit risk analysis.
- Medical diagnosis.
- Face recognition.
- Customer segmentation.

Result:

The Linear Discriminant Analysis (LDA) model was successfully implemented to classify loan applicants into Eligible and Non-Eligible groups using Applicant Income and Credit History. The classifier achieved good accuracy in predicting loan eligibility.

Conclusion:

The Linear Discriminant Analysis (LDA) algorithm effectively classifies loan applicants based on income and credit history. It provides accurate predictions and is suitable for financial institutions to assist in loan approval decisions.

Q5 Apply a probabilistic generative model to classify handwritten digits in an optical character recognition system.

Task:

To apply a Probabilistic Generative Model (Naïve Bayes) to classify handwritten digits (0–9) in an Optical Character Recognition (OCR) system.

Aim:

To implement the Gaussian Naïve Bayes algorithm to classify handwritten digits using pixel values from the MNIST Digits dataset.

Theory:

A Probabilistic Generative Model learns the probability distribution of each class and predicts the class with the highest posterior probability.

Gaussian Naïve Bayes (GNB) assumes that the features are independent and normally distributed within each class.

In OCR systems:

- **Input:** Pixel values of handwritten digit images

- **Output:** Digit (0–9)

Naïve Bayes computes the probability of each digit class and predicts the digit having the highest probability.

Mathematical Formula:

Using Bayes' Theorem:

$$P(C | X) = \frac{P(X | C) \times P(C)}{P(X)}$$

Where

- **P(C|X)** = Posterior Probability
- **P(X|C)** = Likelihood
- **P(C)** = Prior Probability
- **P(X)** = Evidence

Dataset Name:

MNIST Handwritten Digits Dataset

Kaggle Dataset Link:

<https://www.kaggle.com/datasets/oddrational/mnist-in-csv>

Dataset Description:

Feature	Description
label	Digit (0–9)
pixel1–pixel784	Pixel intensity values

- **Target:** label
- **Features:** 784 pixel values

Algorithm Steps:

Step 1: Import the required Python libraries.

Step 2: Upload and load the MNIST dataset.

Step 3: Separate the input features and target label.

Step 4: Split the dataset into training and testing datasets.

Step 5: Create the Gaussian Naïve Bayes classifier.

Step 6: Train the classifier.

Step 7: Predict handwritten digits.

Step 8:Evaluate the model using:

- Accuracy
- Confusion Matrix
- Classification Report

Step 9:Predict a handwritten digit.

Code:

```
# Handwritten Digit Classification using Naive Bayes
# Dataset: mnist_train.csv

import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print(df.head())

X = df.drop('label', axis=1)
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```



```

print("\nAccuracy :", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix")

print(confusion_matrix(y_test, y_pred))

print("\nClassification Report")

print(classification_report(y_test, y_pred))

sample = X_test.iloc[[0]]

prediction = model.predict(sample)

print("\nPredicted Digit :", prediction[0])

print("Actual Digit :", y_test.iloc[0])

```

Output:

```

Choose Files mnist_train[1].csv
... mnist_train[1].csv(text/csv) - 109640201 bytes, last modified: 8/5/2026 - 100% done
Saving mnist_train[1].csv to mnist_train[1].csv
  label  1x1  1x2  1x3  1x4  1x5  1x6  1x7  1x8  1x9  ...  28x19  28x20  \
0      5    0    0    0    0    0    0    0    0    0  ...    0    0
1      0    0    0    0    0    0    0    0    0    0  ...    0    0
2      4    0    0    0    0    0    0    0    0    0  ...    0    0
3      1    0    0    0    0    0    0    0    0    0  ...    0    0
4      9    0    0    0    0    0    0    0    0    0  ...    0    0

      28x21  28x22  28x23  28x24  28x25  28x26  28x27  28x28
0      0      0      0      0      0      0      0      0
1      0      0      0      0      0      0      0      0
2      0      0      0      0      0      0      0      0
3      0      0      0      0      0      0      0      0
4      0      0      0      0      0      0      0      0

[5 rows x 785 columns]

Accuracy : 0.5600833333333334

Confusion Matrix
[[1028    1    9    0    0    2  43    1   68   23]
 [   2 1237    7    8    0    3  12    1   41   11]
 [   73   33  371   86    6    7 290    0  295   13]
 [   58   65   15  466    2    5  55   10  423  120]
 [   26    8   13    6  225    6 104    2  231  555]
 [  124   31    6   22    5   61  52    3  683  117]
 [    9   18    8    2    1    2 1113    0   23    1]
 [    9   11    3   15   13    2    0  353   44  849]
 [   18  145    6    7    3   10   21    1  721  228]
 [    7    3    4    0    5    1    0    9   19 1146]]

```

Classification Report					
	precision	recall	f1-score	support	
0	0.76	0.87	0.81	1175	
1	0.80	0.94	0.86	1322	
2	0.84	0.32	0.46	1174	
3	0.76	0.38	0.51	1219	
4	0.87	0.19	0.31	1176	
5	0.62	0.06	0.10	1104	
6	0.66	0.95	0.78	1177	
7	0.93	0.27	0.42	1299	
8	0.28	0.62	0.39	1160	
9	0.37	0.96	0.54	1194	
accuracy			0.56	12000	
macro avg	0.69	0.56	0.52	12000	
weighted avg	0.69	0.56	0.52	12000	

Predicted Digit : 7
Actual Digit : 7

Advantages:

- Simple and easy to implement.
- Fast training and prediction.
- Suitable for probabilistic classification.
- Efficient for high-dimensional data.
- Performs well as a baseline OCR classifier.

Applications:

- Optical Character Recognition (OCR).
- Handwritten digit recognition.
- Bank cheque processing.
- Postal ZIP code recognition.
- Automatic form processing.

Result:

The Gaussian Naïve Bayes classifier was successfully implemented on the MNIST Handwritten Digits Dataset. The model classified handwritten digits (0–9) with good accuracy and demonstrated effective probabilistic classification for OCR tasks.

Conclusion:

The Gaussian Naïve Bayes algorithm is a simple and effective probabilistic generative model for handwritten digit classification. By learning the probability distribution of pixel values, it

accurately predicts digit classes and serves as a strong baseline for Optical Character Recognition (OCR) systems.

Q6 Implement a probabilistic discriminative model to predict customer churn from service usage data in a telecom scenario.

Task:

To implement a Probabilistic Discriminative Model (Logistic Regression) to predict customer churn based on telecom service usage data.

Aim:

To implement the Logistic Regression algorithm using Python to predict whether a telecom customer is likely to churn (leave the service) based on service usage features.

Theory:

A Probabilistic Discriminative Model directly models the probability of a customer belonging to a particular class (Churn or No Churn). Logistic Regression is one of the most commonly used probabilistic discriminative classifiers.

In telecom companies, customer churn prediction helps identify customers who are likely to discontinue the service.

Input Features

- Monthly Charges
- Tenure
- Total Charges

Target

- Churn (Yes/No)

Mathematical Formula:

The Logistic Regression model is

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

where

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

Where

- Y = Churn Prediction
- β_0 = Intercept
- $\beta_1, \beta_2, \beta_3$ = Model Coefficients

- X_1 = Monthly Charges
- X_2 = Tenure
- X_3 = Total Charges

Dataset Name:

Telco Customer Churn Dataset

Kaggle Dataset Link:

<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

Dataset Description:

Feature	Description
tenure	Customer subscription period
MonthlyCharges	Monthly bill amount
TotalCharges	Total bill amount
Churn	Yes/No (Target)

Algorithm Steps:

Step 1: Import the required Python libraries.

Step 2: Upload and load the Telco Customer Churn dataset.

Step 3: Select the required columns:

- tenure
- MonthlyCharges
- TotalCharges
- Churn

Step 4: Remove missing values.

Step 5: Convert:

- Churn $\rightarrow$ Yes = 1, No = 0

Step 6: Split the dataset into training and testing datasets.

Step 7: Train the Logistic Regression classifier.

Step 8: Predict customer churn.

Step 9: Evaluate the model using:

- Accuracy

- Confusion Matrix
- Classification Report

Step 10: Predict churn for a new customer.

Code:

```
# Customer Churn Prediction using Logistic Regression
# Dataset: WA_Fn-UseC_-Telco-Customer-Churn.csv

import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print(df.head())

df = df[['tenure',
        'MonthlyCharges',
        'TotalCharges',
        'Churn']]

df['TotalCharges'] = pd.to_numeric(df['TotalCharges'],
                                   errors='coerce')

df = df.dropna()

df['Churn'] = df['Churn'].map({
    'Yes':1,
    'No':0
})

X = df[['tenure',
        'MonthlyCharges',
```

```

        'TotalCharges']]
y = df['Churn']
X_train,X_test,y_train,y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
model = LogisticRegression(max_iter=1000)
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("\nAccuracy :",accuracy_score(y_test,y_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test,y_pred))
print("\nClassification Report")
print(classification_report(y_test,y_pred))
# Predict New Customer
# tenure=24 months
# MonthlyCharges=75
# TotalCharges=1800
new_customer = pd.DataFrame({
    'tenure':[24],
    'MonthlyCharges':[75],
    'TotalCharges':[1800]
})
prediction = model.predict(new_customer)
if prediction[0] == 1:
    print("\nPrediction : Customer Will Churn")
else:
    print("\nPrediction : Customer Will Not Churn")

```

Output:

... Choose Files WA_Fn-Us...Churn[1].csv
WA_Fn-UseC_-Telco-Customer-Churn[1].csv(text/csv) - 977501 bytes, last modified: 8/5/2026 - 100% done
Saving WA_Fn-UseC_-Telco-Customer-Churn[1].csv to WA_Fn-UseC_-Telco-Customer-Churn[1].csv

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	

	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	\
0	No phone service	DSL	No	...	No	
1	No	DSL	Yes	...	Yes	
2	No	DSL	Yes	...	No	
3	No phone service	DSL	Yes	...	Yes	
4	No	Fiber optic	No	...	No	

	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	\
0	No	No	No	Month-to-month	Yes	
1	No	No	No	One year	No	
2	No	No	No	Month-to-month	Yes	
3	Yes	No	No	One year	No	
4	No	No	No	Month-to-month	Yes	

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes

Confusion Matrix

```
[[937 96]
 [214 160]]
```

Classification Report

	precision	recall	f1-score	support
0	0.81	0.91	0.86	1033
1	0.62	0.43	0.51	374
accuracy			0.78	1407
macro avg	0.72	0.67	0.68	1407
weighted avg	0.76	0.78	0.76	1407

Prediction : Customer Will Not Churn

Advantages:

- Simple and easy to implement.
- Produces probability estimates.
- Fast training and prediction.
- Suitable for binary classification.
- Performs well on telecom customer datasets.
- Easy to interpret.

Applications:

- Customer churn prediction.
- Telecom analytics.
- Banking customer retention.
- Insurance policy renewal prediction.
- Subscription service analysis.

Result:

The Logistic Regression model was successfully implemented using the Telco Customer Churn Dataset. The model accurately predicted whether a customer would churn or remain with the telecom service based on tenure, monthly charges, and total charges.

Conclusion:

The Logistic Regression algorithm effectively predicts customer churn in a telecom environment. By analyzing customer service usage features, it estimates the probability of churn and helps telecom companies identify at-risk customers, enabling proactive retention strategies.

Q7 Apply Bayesian logistic regression to predict whether a customer will purchase a product after viewing online advertisements.**Task:**

To apply Bayesian Logistic Regression to predict whether a customer will purchase a product after viewing online advertisements.

Aim:

To implement a Bayesian Logistic Regression model using Python to predict customer purchase decisions based on online advertisement data.

Theory:

Bayesian Logistic Regression is a probabilistic classification algorithm that combines Logistic Regression with Bayesian inference. Instead of estimating a single value for each model parameter, Bayesian Logistic Regression estimates a probability distribution over the parameters.

This approach provides:

- Probability of customer purchase.
- Better uncertainty estimation.
- Improved robustness for small datasets.

In online advertising:

- **Input Features**
 - Age
 - Estimated Salary
- **Target**
 - Purchased (Yes/No)

Mathematical Formula:

The logistic function is

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

where

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

Where

- **Y** = Purchase Prediction
- **β_0** = Intercept
- **β_1, β_2** = Model Parameters
- **X_1** = Age
- **X_2** = Estimated Salary

Dataset Name:

Social Network Ads Dataset

Kaggle Dataset Link:

<https://www.kaggle.com/datasets/rakeshrau/social-network-ad>

Dataset Description:

Feature	Description
Age	Customer Age
EstimatedSalary	Estimated Salary
Purchased	Product Purchased (0/1)

Algorithm Steps:

Step 1: Import the required libraries.

Step 2: Upload and load the dataset.

Step 3: Select the required features:

- Age
- EstimatedSalary

Target:

- Purchased

Step 4: Split the dataset into training and testing datasets.

Step 5: Standardize the feature values.

Step 6: Train the Bayesian Logistic Regression model.

Step 7: Predict purchase decisions.

Step 8: Evaluate using:

- Accuracy
- Confusion Matrix
- Classification Report

Step 9: Predict purchase for a new customer.

Code:

```
import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import BayesianRidge
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print(df.head())
df = df[['Age', 'EstimatedSalary', 'Purchased']]
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = BayesianRidge()
model.fit(X_train, y_train)
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob >= 0.5).astype(int)
print("\nAccuracy :", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report")
print(classification_report(y_test, y_pred))
# Predict New Customer
```

```
# Age = 35

# Estimated Salary = 65000

new_customer = scaler.transform([[35,65000]])

prediction = model.predict(new_customer)

if prediction[0] >= 0.5:

    print("\nPrediction : Customer Will Purchase")

else:

    print("\nPrediction : Customer Will Not Purchase")
```

Output:

```
... Choose Files Social_Net...k_Ads[1].csv
Social_Network_Ads[1].csv(text/csv) - 10926 bytes, last modified: 8/5/2026 - 100% done
Saving Social_Network_Ads[1].csv to Social_Network_Ads[1].csv
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510   Male   19             19000         0
1  15810944   Male   35             20000         0
2  15668575  Female   26             43000         0
3  15603246  Female   27             57000         0
4  15804002   Male   19             76000         0

Accuracy : 0.8625

Confusion Matrix
[[50  2]
 [ 9 19]]

Classification Report
      precision    recall  f1-score   support

      0       0.85      0.96      0.90         52
      1       0.90      0.68      0.78         28

   accuracy              0.86         80
  macro avg              0.88         80
 weighted avg              0.87         80

Prediction : Customer Will Not Purchase
```

Advantages:

- Provides probabilistic predictions.
- Handles uncertainty effectively.
- Reduces overfitting.

- Performs well on marketing datasets.
- Suitable for binary classification.

Applications:

- Online advertisement analytics.
- Customer purchase prediction.
- Digital marketing.
- Recommendation systems.
- Customer behavior analysis.

Result:

The Bayesian Logistic Regression model was successfully implemented to predict whether a customer would purchase a product after viewing online advertisements. The model achieved good classification accuracy and produced reliable probabilistic predictions.

Conclusion:

The Bayesian Logistic Regression model effectively predicts customer purchase behavior based on features such as Age and Estimated Salary. By incorporating Bayesian inference, it provides probabilistic estimates that help marketers make informed decisions and improve targeted advertising strategies.

Q8 Use linear regression to forecast monthly sales for a retail company based on advertising expenditure and seasonal demand.

Task

To use Linear Regression to forecast monthly sales for a retail company based on Advertising Expenditure and Seasonal Demand.

Aim:

To implement a Multiple Linear Regression model using Python to predict monthly sales using advertising expenditure and seasonal demand data.

Theory:

Linear Regression is a supervised machine learning algorithm used to predict continuous values. It establishes a linear relationship between one dependent variable and one or more independent variables.

In this experiment:

- **Input Features**

- Advertising Expenditure
- Seasonal Demand
- **Target Variable**
 - Monthly Sales

The model learns the relationship between advertising investment, seasonal demand, and sales to forecast future monthly sales.

Mathematical Formula:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

Where

- **Y** = Monthly Sales
- **β_0** = Intercept
- **β_1** = Coefficient for Advertising Expenditure
- **β_2** = Coefficient for Seasonal Demand
- **X_1** = Advertising Expenditure
- **X_2** = Seasonal Demand

Dataset Name:

Advertising Sales Dataset

Kaggle Dataset Link:

<https://www.kaggle.com/datasets/ashydv/advertising-dataset>

Dataset Description:

Feature	Description
TV	TV Advertising Expenditure
Radio	Radio Advertising Expenditure
Newspaper	Newspaper Advertising Expenditure
Sales	Product Sales

For this experiment:

- Advertising Expenditure = TV
- Seasonal Demand = Radio
- Target = Sales

Algorithm Steps:

Step 1: Import the required Python libraries.

Step 2: Upload and load the Advertising dataset.

Step 3: Select:

- TV
- Radio
- Sales

Step 4: Split the dataset into training and testing datasets.

Step 5: Train the Multiple Linear Regression model.

Step 6: Predict monthly sales.

Step 7: Evaluate using:

- MAE
- MSE
- RMSE
- R^2 Score

Step 8: Predict sales for new advertising expenditure values.

Code:

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print(df.head())
X = df[['TV','Radio']]
y = df['Sales']
```

```

X_train,X_test,y_train,y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
model = LinearRegression()

model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("\nMAE :",mean_absolute_error(y_test,y_pred))
print("MSE :",mean_squared_error(y_test,y_pred))
print("RMSE :",np.sqrt(mean_squared_error(y_test,y_pred)))
print("R2 Score :",r2_score(y_test,y_pred))
# Predict New Sales
# TV = 250
# Radio = 40
new_data = pd.DataFrame({
    'TV':[250],
    'Radio':[40]
})
prediction = model.predict(new_data)
print("\nPredicted Monthly Sales =",prediction[0])
print("\nIntercept =",model.intercept_)
print("\nCoefficients")
for feature,coef in zip(X.columns,model.coef_):
    print(feature,"=",coef)

```


Output:

```
... Choose Files advertising[1].csv
advertising[1].csv(text/csv) - 4062 bytes, last modified: 8/5/2026 - 100% done
Saving advertising[1].csv to advertising[1].csv
      TV  Radio  Newspaper  Sales
0  230.1   37.8         69.2   22.1
1   44.5   39.3         45.1   10.4
2   17.2   45.9         69.3   12.0
3  151.5   41.3         58.5   16.5
4  180.8   10.8         58.4   17.9

MAE : 1.2670422434730484
MSE : 2.846616122131541
RMSE : 1.6871917858179433
R2 Score : 0.9078797802624651

Predicted Monthly Sales = 22.548527602326555

Intercept = 4.791381661776025

Coefficients
TV = 0.05450736175135675
Radio = 0.10325763756778356
```

Advantages:

- Simple and easy to implement.
- Fast model training.
- Suitable for continuous-value prediction.
- Easy to interpret.
- Helps in business sales forecasting.

Applications:

- Retail sales forecasting.
- Marketing budget planning.
- Demand forecasting.
- Revenue prediction.
- Business analytics.

Result:

The Multiple Linear Regression model was successfully implemented to forecast monthly sales based on advertising expenditure and seasonal demand. The model achieved good prediction accuracy and effectively estimated future sales.

Conclusion:

The Linear Regression model effectively predicts monthly retail sales using advertising expenditure and seasonal demand. It enables businesses to estimate future sales, optimize marketing budgets, and make data-driven decisions for improved business performance.

Q9 Apply Naïve Bayes classification in a news portal to categorize articles into sports, politics, and entertainment.**Task:**

To apply the Naïve Bayes Classification algorithm in a news portal to categorize news articles into Sports, Politics, and Entertainment.

Aim:

To implement the Multinomial Naïve Bayes algorithm using Python to classify news articles into Sports, Politics, and Entertainment categories.

Theory:

Naïve Bayes is a supervised machine learning algorithm based on Bayes' Theorem. It is widely used for text classification because it is simple, fast, and performs well on textual data.

In this experiment:

Input: News article text

Output: Category

- Sports
- Politics
- Entertainment

The article text is converted into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency), and the Naïve Bayes classifier predicts the most probable category.

Mathematical Formula:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)} = \frac{0.85(0.20)}{0.25} = 0.68$$

$$P(A)$$

$$P(B | A)$$

$$P(B | A)$$

$$P(B | \neg A)$$

$$P(B | \neg A)$$

$P(B)=0.25$ $P(B|A)P(A)=0.17$ $P(A|B)\sim0.68$ Posterior = useful evidence / total evidence

$$P(C | X) = \frac{P(X | C) \times P(C)}{P(X)}$$

Where

- **P(C|X)** = Posterior Probability
- **P(X|C)** = Likelihood
- **P(C)** = Prior Probability
- **P(X)** = Evidence

Dataset Name:

News Category Dataset

Kaggle Dataset Link:

<https://www.kaggle.com/datasets/rmisra/news-category-dataset>

Dataset Description:

Feature	Description
category	News Category
headline	News Headline
short_description	News Description

For this experiment, use:

- **headline + short_description** → Input Text
- **category** → Target

Filter the dataset to only these categories:

- Sports
- Politics

- Entertainment

Algorithm Steps:

Step 1: Import the required Python libraries.

Step 2: Upload and load the News Category dataset.

Step 3: Filter the dataset to include only:

- Sports
- Politics
- Entertainment

Step 4: Combine the headline and short description into a single text field.

Step 5: Convert text into TF-IDF features.

Step 6: Split the dataset into training and testing datasets.

Step 7: Train the Multinomial Naïve Bayes classifier.

Step 8: Predict article categories.

Step 9: Evaluate using:

- Accuracy
- Confusion Matrix
- Classification Report

Step 10: Predict the category of a new news article.

Code:

```
import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

uploaded = files.upload()
filename = list(uploaded.keys())[0]
```

```

df = pd.read_json(filename, lines=True)
print(df.head())
categories = ['SPORTS','POLITICS','ENTERTAINMENT']
df = df[df['category'].isin(categories)]
df['text'] = df['headline'] + " " + df['short_description']
X = df['text']
y = df['category']
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(X)
X_train,X_test,y_train,y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
model = MultinomialNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("\nAccuracy :",accuracy_score(y_test,y_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test,y_pred))
print("\nClassification Report")
print(classification_report(y_test,y_pred))
article = ["India wins the cricket world cup after a thrilling final match."]
article_vector = vectorizer.transform(article)
prediction = model.predict(article_vector)

print("\nPredicted Category :",prediction[0])

```

Output:

```
Choose Files News_Cate...t_v3[1].json
... News_Category_Dataset_v3[1].json(application/json) - 87295572 bytes, last modified: 8/5/2026 - 100% done
Saving News_Category_Dataset_v3[1].json to News_Category_Dataset_v3[1].json
link \
0 https://www.huffpost.com/entry/covid-boosters-...
1 https://www.huffpost.com/entry/american-airlin...
2 https://www.huffpost.com/entry/funniest-tweets...
3 https://www.huffpost.com/entry/funniest-parent...
4 https://www.huffpost.com/entry/amy-cooper-lose...

                                headline      category \
0 Over 4 Million Americans Roll Up Sleeves For O... U.S. NEWS
1 American Airlines Flyer Charged, Banned For Li... U.S. NEWS
2 23 Of The Funniest Tweets About Cats And Dogs ... COMEDY
3 The Funniest Tweets From Parents This Week (Se... PARENTING
4 Woman Who Called Cops On Black Bird-Watcher Lo... U.S. NEWS

                                short_description      authors \
0 Health experts said it is too early to predict... Carla K. Johnson, AP
1 He was subdued by passengers and crew when he ... Mary Papenfuss
2 "Until you have a dog you don't understand wha... Elyse Wanshel
3 "Accidentally put grown-up toothpaste on my to... Caroline Bologna
4 Amy Cooper accused investment firm Franklin Te... Nina Golgowski

                                date
0 2022-09-23
1 2022-09-23
2 2022-09-23
3 2022-09-23
4 2022-09-22

Accuracy : 0.8876733568782841

Classification Report
                                precision      recall      f1-score      support

ENTERTAINMENT      0.91      0.85      0.87      3477
POLITICS      0.88      0.99      0.93      7138
SPORTS      0.98      0.31      0.47      994

                                accuracy
                                macro avg      0.92      0.71      0.76      11609
                                weighted avg      0.89      0.89      0.87      11609

Predicted Category : SPORTS
```

Advantages:

- Fast and efficient for text classification.
- Easy to implement.
- Performs well on large text datasets.
- Handles multi-class classification.

- Requires less training time.

Applications:

- News article classification.
- Email spam filtering.
- Sentiment analysis.
- Document categorization.
- Topic detection.

Result:

The Multinomial Naïve Bayes classifier was successfully implemented to classify news articles into Sports, Politics, and Entertainment categories. The model achieved high classification accuracy using TF-IDF text features.

Conclusion:

The Multinomial Naïve Bayes algorithm effectively classifies news articles into multiple categories by learning the probability distribution of words in each category. Its simplicity, speed, and high accuracy make it well suited for real-world news portals and text classification applications.

Q10. Use Bayesian Logistic Regression to predict student placement outcomes based on CGPA, coding skills, and aptitude scores.**Task:**

To apply Bayesian Logistic Regression to predict student placement outcomes based on CGPA, Coding Skills, and Aptitude Scores.

Aim:

To implement a Bayesian Logistic Regression model using Python to predict whether a student will be Placed or Not Placed based on academic and skill-based features.

Theory:

Bayesian Logistic Regression is a probabilistic classification algorithm that combines Logistic Regression with Bayesian inference. Unlike traditional Logistic Regression, Bayesian Logistic Regression estimates a probability distribution for model parameters, providing uncertainty estimates and improving robustness.

In this experiment:

Input Features

- CGPA
- Coding Skills Score
- Aptitude Score

Target

- Placement (Placed / Not Placed)

Mathematical Formula:

The logistic function is

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

where

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

Where

- Y = Placement Prediction
- β_0 = Intercept
- β_1 = CGPA Coefficient
- β_2 = Coding Skills Coefficient
- β_3 = Aptitude Score Coefficient

Dataset Name:

Campus Placement Prediction Dataset

Kaggle Dataset Link:

<https://www.kaggle.com/datasets/benroshan/factors-affecting-campus-placement>

Dataset Description:

Feature	Description
cgpa	Student CGPA
aptitude_score	Aptitude Test Score
coding_score	Coding Skills Score
placement	Placed (1) / Not Placed (0)

Algorithm Steps:

Step 1: Import the required Python libraries.

Step 2: Upload and load the placement dataset.

Step 3: Select the required features:

- CGPA
- Coding Score
- Aptitude Score

Target:

- Placement

Step 4: Remove missing values.

Step 5: Split the dataset into training and testing datasets.

Step 6: Standardize the feature values.

Step 7: Train the Bayesian Logistic Regression model.

Step 8: Predict placement outcomes.

Step 9: Evaluate using:

- Accuracy
- Confusion Matrix
- Classification Report

Step 10: Predict placement for a new student.

Code:

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import BayesianRidge
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
uploaded = files.upload()
```

```

filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print(df.head())
df = df[['degree_p','etest_p','mba_p','status']]
df.columns = ['CGPA','CodingScore','AptitudeScore','Placement']
df['Placement'] = df['Placement'].map({
    'Placed':1,
    'Not Placed':0
})
# Remove Missing Values
df.dropna(inplace=True)
X = df[['CGPA','CodingScore','AptitudeScore']]
y = df['Placement']
X_train,X_test,y_train,y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = BayesianRidge()
model.fit(X_train,y_train)
y_prob = model.predict(X_test)
y_pred = (y_prob >= 0.5).astype(int)
print("\nAccuracy :",accuracy_score(y_test,y_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test,y_pred))
print("\nClassification Report")

```

```

print(classification_report(y_test,y_pred))

# degree_p = 70

# etest_p = 75

# mba_p = 68

new_student = scaler.transform([[70,75,68]])

prediction = model.predict(new_student)

if prediction[0] >= 0.5:

    print("\nPrediction : Student Will Be Placed")

else:

    print("\nPrediction : Student Will Not Be Placed")

print("\nIntercept :", model.intercept_)

print("\nCoefficients")

for feature, coef in zip(X.columns, model.coef_):

    print(feature, "=", coef)

```

Output:

```

... Choose Files Placement_...ll_Class.csv
Placement_Data_Full_Class.csv(text/csv) - 19712 bytes, last modified: 8/5/2026 - 100% done
Saving Placement_Data_Full_Class.csv to Placement_Data_Full_Class (1).csv
  sl_no gender  ssc_p  ssc_b  hsc_p  hsc_b  hsc_s  degree_p  \
0      1      M  67.00  Others  91.00  Others  Commerce  58.00
1      2      M  79.33  Central  78.33  Others  Science    77.48
2      3      M  65.00  Central  68.00  Central  Arts       64.00
3      4      M  56.00  Central  52.00  Central  Science    52.00
4      5      M  85.80  Central  73.60  Central  Commerce   73.30

  degree_t workex  etest_p specialisation  mba_p  status  salary
0  Sci&Tech     No   55.0      Mkt&HR    58.80  Placed  270000.0
1  Sci&Tech    Yes   86.5      Mkt&Fin    66.28  Placed  200000.0
2  Comm&Mgmt   No   75.0      Mkt&Fin    57.80  Placed  250000.0
3  Sci&Tech     No   66.0      Mkt&HR    59.43  Not Placed  NaN
4  Comm&Mgmt   No   96.8      Mkt&Fin    55.50  Placed  425000.0

Accuracy : 0.7209302325581395

Confusion Matrix
[[ 3  9]
 [ 3 28]]

Classification Report
              precision    recall  f1-score   support

0               0.50         0.25         0.33         12
1               0.76         0.90         0.82         31

```

accuracy			0.72	43
macro avg	0.63	0.58	0.58	43
weighted avg	0.69	0.72	0.69	43

Prediction : Student Will Be Placed

Intercept : 0.6802325581395348

Coefficients

CGPA = 0.23627752312993705

CodingScore = 0.02748872963127779

AptitudeScore = -0.04195378590352911

Advantages:

- Provides probabilistic predictions.
- Handles uncertainty effectively.
- Reduces overfitting.
- Suitable for binary classification.
- Improves decision-making with probability estimates.

Applications:

- Campus placement prediction.
- Student performance analysis.
- Recruitment analytics.
- HR candidate screening.
- Educational data mining.

Result:

The Bayesian Logistic Regression model was successfully implemented to predict student placement based on CGPA, Coding Skills, and Aptitude Scores. The model accurately classified students into Placed and Not Placed categories.

Conclusion:

The Bayesian Logistic Regression model effectively predicts student placement outcomes using academic and technical skill features. It provides probabilistic predictions that help educational institutions and recruiters make informed placement decisions.